Table of Contents

1. Complex queries from username and password: 2
2. Time Handling: 4
3. Button handling in selecting week: 7
4. Construct Table for User Schedule 9
5. Handling User Schedule data storage: 12
6. Handling Week Data extraction: 13
7. Allow Admin to output schedule as a HTML file: 15
8. Schedule Timetable: 17
9. Libraries used: 24

Bibliography: 25

1. Complex queries from username and password:

The following code in the frame “Welcome” which allows program to turn the TXT file that used to store User or Admin’s password and username into a ArrayList as the program doesn’t know how much data there is. This program is implemented for login system to fetch all the login detail for the system to recognize and whether allow the user or admin to login or not:

```
private List<String> getList(String path) {
    //User dynamic data structure Array List to append data
    List<String> list = new ArrayList<>();
    //user buffered reader from java.io to extract data from txt file
    //user try and catch to handle exception
    try (BufferedReader reader = new BufferedReader(new FileReader(path))) {
        String line;
        /* write user or admin Password into Array split with the character
         "|" and handle exception*/
        while ((line = reader.readLine()) != null) {
            String[] items = line.split("\\|");
            for (String item : items) {
                if (!item.trim().isEmpty()) {
                    list.add(item.trim());
                }
            }
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
    return list;
}
```

[1]*Figure 1.0 – getList() method*

This method check() is used to determine whether the User or the Admin is allowed to login. In this method, it compares the entered email to all email in the stored file and check if the correspond password is correct or not. This is used as an external method to keep original code clean:

[1] GeeksforGeeks. “Java.io.BufferedReader Class Methods in Java.” GeeksforGeeks, December 29, 2016. https://www.geeksforgeeks.org/io-bufferedreader-class-methods-java.

```java
public boolean check(String emailPath, String passwordPath, String passwordEntered, String emailEntered) {
    List<String> emailList = getList(emailPath);
    List<String> passwordList = getList(passwordPath);
    //check if the entered email and password match with any of the registered email and password
    for (int i = 0; i < emailList.size(); i++) {
        String storedEmail = emailList.get(i);
        String storedPassword = passwordList.get(i);
        if (emailEntered.equals(storedEmail) && passwordEntered.equals(storedPassword)) {
            return true;
        }
    }
    return false;
}
```

Figure 1.1 – check() method

When the login button is clicked, it checks if login is allowed by using those 2 method shown above (figure 1.0 and figure 1.1). This ActionListener is implemented to detect the login button from user and check if valid. It also separates admin from user:

```java
else if(e.getSource() == Login) {
    //get user input (use character array to keep the password textfield private)
    char[] passwordchararray = PasswordEnter.getPassword();
    String password = new String(passwordchararray);
    INPUT = Email.getText();
    //check if login detail accepted (refer to figure 1.1 check() method)
    boolean valid = check(AdminEmaildocumentpath, AdminPassworddocumentpath, password, INPUT)
            || check(Emaildocumentpath, Passworddocumentpath, password, INPUT);
    if(valid) {
        //differentiate between User and Admin
        User currentuser = User.getuser(INPUT);
        if(currentuser.isAdmin()) {
            //open an Admin Main Frame
            Admin currentadmin = Admin.getadmin(INPUT);
            new AdminMainFrame(currentadmin);
        }
        else {
            //open an User Main Frame
            new MainFrame(currentuser);
        }
        //close Welcome Page
        this.dispose();
    }
    else {
        //display input error
        JOptionPane.showOptionDialog(
                null,
                "incorrect email or password",
                "error",
                JOptionPane.DEFAULT_OPTION,
                JOptionPane.ERROR_MESSAGE,
                null,
                new Object[] { "OK" },
                "OK"
            );
    }
}
```

Figure 1.2 – ActionListener for Login Button

In the register page. The system needs to register the password and email of a User (same principle is applied in the Admin Register page, but path of document is different):

```
@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == Registerbutton) {
        String registerEmail = Email.getText().trim();
        char[] passwordchararray = PasswordEnter.getPassword();
        String password = new String(passwordchararray);
        //check if email is already registered in the data
        if (isEmailRegistered(registerEmail)) {
            //If registered display frame of error message
            JOptionPane.showMessageDialog(this,
                    "This email is already registered!", "Error", JOptionPane.ERROR_MESSAGE);
        } else {
            //use Buffered Writer to write the register email and password
            try (BufferedWriter emailWriter = new BufferedWriter(new FileWriter(Emaildocumentpath, true))) {
                emailWriter.write(registerEmail + "|");
                emailWriter.newLine();
            } catch (IOException e1) {
                e1.printStackTrace();
            }
            try (BufferedWriter passwordWriter = new BufferedWriter(new FileWriter(Passworddocumentpath, true))) {
                passwordWriter.write(password + "|");
                passwordWriter.newLine();
            } catch (IOException e1) {
                e1.printStackTrace();
            }
            //close Register frame
            this.dispose();
            //back to Welcome frame
            new User(registerEmail, password);
            new Welcome();
        }
    }
}
```
[2]

Figure 1.3 – ActionListener for Register Button

2. Time Handling:

In the Page of Choosing week for Both User and Admin, the system should be able to calculate the time of different week start date from local time by using formula such as:
Current Week Starting date = time now - (numerical day of today – 1)

Next week Starting date = time now + 7 - (numerical day of today – 1)

Which is combined with the java.time library.

This is implemented to find each start of week date, which is easier to access files and organize files.

[2] GeeksforGeeks. "Java.io.BufferedWriter Class Methods in Java." GeeksforGeeks, December 29, 2016. https://www.geeksforgeeks.org/io-bufferedwriter-class-methods-java.

```java
@Override
public void actionPerformed(ActionEvent e) {
    if(e.getSource() == goBackButton) {
        this.dispose();
    }
    if(e.getSource() == currentWeekButton) {
        //use java.time to calculate the start of current week
        LocalDate today = LocalDate.now();
        DayOfWeek dayinweek = today.getDayOfWeek();
        int daynumber = dayinweek.getValue();
        LocalDate startofthisweek = today.minusDays(daynumber - 1);
        String Startofcurrentweekdate = startofthisweek.toString();
        new UserCurrentweekpage(Startofcurrentweekdate, user);
    }
    if(e.getSource() == nextWeekButton) {
        //use java.time to calculate the start of next week
        LocalDate today = LocalDate.now();
        DayOfWeek dayinweek = today.getDayOfWeek();
        int daynumber = dayinweek.getValue();
        LocalDate startofnextweek = today.plusDays(7);
        startofnextweek = startofnextweek.minusDays(daynumber - 1);
        String Startofcurrentweekdate = startofnextweek.toString();
        new UserCurrentweekpage(Startofcurrentweekdate, user);
    }
}
```

Figure 2.0 – Time handling formula

In the Choose week page class, we also need to generate week data for 4 different consecutive weeks (Refer to Criterion B – Design of choose week page) with following method:

```java
private void generateScheduleWeekData() {
    //use java.text.Simple Date Format to format date
    SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
    //get start of date of next week
    Calendar calendar = Calendar.getInstance();
    LocalDate today = LocalDate.now();
    DayOfWeek dayinweek = today.getDayOfWeek();
    int daynumber = dayinweek.getValue();
    today = today.plusDays(7);
    today = today.minusDays(daynumber - 1);
    int year = today.getYear();
    int month = today.getMonthValue() - 1;
    int day = today.getDayOfMonth();
    //convert date into calendar (month-1 due to calendar uses 0 based months)
    calendar.set(year, month, day);
    //generate 4 weeks of schedule data
    for (int i = 1; i <= 4; i++) {
        String date = sdf.format(calendar.getTime());
        int weekNumber = i;
        String abWeek = (i % 2 == 1) ? "A" : "B";
        JButton button = new JButton("-> Schedule");
        tableModel.addRow(new Object[]{date, weekNumber, abWeek, button});
        calendar.add(Calendar.DAY_OF_MONTH, 7);
    }
}
```

Figure 2.1 – generateScheduleWeekData() method

3. Button handling in selecting week:

In the Choose week page the system uses 2 extra classes of ButtonEditor and ButtonRenderer that handles button in a JTable because original java class does not allow customize position of buttons in table. Therefore, we need external created class to help organize format

The class enables interactive buttons within a cell and enables dynamic to integrate of button into the GUI.

```
private class ButtonEditor extends DefaultCellEditor {
    /*extends Default Cell Editor
     * (responsible for editing cell when a user
     * interact with it inside a JTable*/
    private JButton button;
    private String date;
    private int weekNumber;
    private String weekAB;
    public ButtonEditor(JCheckBox checkBox) {
        super(checkBox);
        button = new JButton();
        button.setOpaque(true);
        /*add actionListener to the button to
         * allow it to open a schedule table page for user with those specific data*/
        button.addActionListener(e -> {
            new Scheduletablepage(date, weekNumber, weekAB, user);
        });
    }
    /*check if the component is a button and Retrieves data from
    row() by using the function .getValueAt().*/
    @Override
    public Component getTableCellEditorComponent(JTable table,
            Object value, boolean isSelected, int row, int column) {
        if (value instanceof JButton) {
            button.setText(((JButton) value).getText());
            date = (String) table.getValueAt(row, 0);
            weekNumber = (Integer) table.getValueAt(row, 1);
            weekAB = (String) table.getValueAt(row, 2);
        }
        //return the button
        return button;
    }
    @Override
    public Object getCellEditorValue() {
        return button;
    }
}
```

Figure 3.0 – ButtonEditor Class

Button Renderer class is another essential component because JTable in default only displays text in cells. But a renderer class enables it to display a clickable button within JTable.

```java
private static class ButtonRenderer extends JButton
implements javax.swing.table.TableCellRenderer {
    //extends javax.swing.table.TableCellRenderer to enable display
    public ButtonRenderer() {
        //visible button
        setOpaque(true);
    }
    /*a method called whenever a button is displayed and
     * determines which component will be shown in the table*/
    @Override
    public Component getTableCellRendererComponent(JTable table,
            Object value, boolean isSelected, boolean hasFocus,
            int row, int column) {
        if (value instanceof JButton) {
            return (JButton) value;
        }
        return this;
    }
}
```

Figure 3.1 – ButtonRenderer class

4. Construct Table for User Schedule

On the user schedule page, the system should be able to display a table for User to enter availability. (Refer to Criterion A – Success Criteria). Therefore in the following code we customize a JTable to fulfill the extraction of user data (refer to Criterion B – Design of user schedule table page)

```
//row and column headers
String[] days = {"Monday", "Tuesday", "Wednesday", "Thursday",
        "Friday", "Saturday", "Sunday"};
String[] times = {"Breakfast", "1st Lunch", "2nd Lunch (Downstairs)",
        "2nd Lunch (Stairs)", "Supper"};
timeSlots = new String[times.length][days.length];
/*table model is initialized with rows corresponding to times.lengh
 * and columns corresponding to days.length+1
 * (additional column for time slot labels)*/
tableModel = new DefaultTableModel(times.length, days.length + 1) {
    @Override
    public boolean isCellEditable(int row, int column) {
        if (column == 7 && (row == 0 || row == 2 || row == 3)) {
            return false;
        }
        return false;
    }
};
//set column headers for table
tableModel.setColumnIdentifiers(
        new String[]{"", "Monday", "Tuesday", "Wednesday",
                "Thursday", "Friday", "Saturday", "Sunday"});
/*Populates Jtable with initial values, setting time slots
 * int he first column and marking it "Available" across the grid*/
for (int i = 0; i < times.length; i++) {
    tableModel.setValueAt(times[i], i, 0);
    for (int j = 1; j <= days.length; j++) {
        if (j == 7 && (i == 0 || i == 2 || i == 3)) {
            tableModel.setValueAt("", i, j);
        } else {
            tableModel.setValueAt("Available", i, j);
        }
    }
}
```

[3]

```java
scheduleTable = new JTable(tableModel);
scheduleTable.setRowHeight(80);
//modify how data is displayed in table cells
DefaultTableCellRenderer centerRenderer = new DefaultTableCellRenderer();
centerRenderer.setHorizontalAlignment(JLabel.CENTER);
//applies a custom cell renderer (refer to figure 4.1) to the table
scheduleTable.setDefaultRenderer(Object.class, new CustomCellRenderer());
//add a mouse listener when a user clicks on certain cell in the table
scheduleTable.addMouseListener(new java.awt.event.MouseAdapter() {
    public void mousePressed(java.awt.event.MouseEvent evt) {
        int row = scheduleTable.rowAtPoint(evt.getPoint());
        int col = scheduleTable.columnAtPoint(evt.getPoint());
        /*prevent user from clicking extra time slot on Sunday*/
        if (col == 7 && (row == 0 || row == 2 || row == 3)) {
            return;
        }
        /* Toggle between "Unavailable" and "Available" for user when they are clicking
         * on the table representing their availability on that specific time slot
         * (Refer to Criterion A - Success Criteria)
         * (Refer to Criterion B - Design of Table)*/
        if (col > 0 && row >= 0) {
            String currentValue = (String) tableModel.getValueAt(row, col);
            if ("Available".equals(currentValue)) {
                tableModel.setValueAt("Unavailable", row, col);
            } else {
                tableModel.setValueAt("Available", row, col);
            }
            scheduleTable.repaint();
        }
    }
});
```

Figure 4.0 – Table construction for User

[3] Justiciar. "Creating a Custom Table Model to Provide Data from ArrayList." Stack Overflow, March 30, 2017. https://stackoverflow.com/questions/43106351/creating-a-custom-table-model-to-provide-data-from-arraylist.

The CustomCellRenderer class:

```
private class CustomCellRenderer extends DefaultTableCellRenderer {
    @Override//override the superclass to get default rendering for the cell
    public Component getTableCellRendererComponent(JTable table,
            Object value, boolean isSelected, boolean hasFocus,
            int row, int column) {
        Component cell = super.getTableCellRendererComponent(
                table, value, isSelected, hasFocus, row, column);

        if (column == 7 && (row == 0 || row == 2 || row == 3)) {
            //if sunday and unchangeable time slot set it always to grey
            cell.setBackground(Color.LIGHT_GRAY);
            cell.setForeground(Color.DARK_GRAY);
        } else if ("Available".equals(value)) {
            //if available turn green
            cell.setBackground(Color.GREEN);
            cell.setForeground(Color.BLACK);
        } else if ("Unavailable".equals(value)) {
            //if unavailable turn red
            cell.setBackground(Color.RED);
            cell.setForeground(Color.WHITE);
        } else {
            //default colour
            cell.setBackground(Color.WHITE);
            cell.setForeground(Color.BLACK);
        }
        return cell;
    }
}
```

Figure 4.1 – CustomCellRenderer class

5. Handling User Schedule data storage:

After User has successfully schedulled, the data should be saved in a file in the ScheduleTablePage class (Refer to Criterion B – Design):

```java
@Override
public void actionPerformed(ActionEvent e) {
    if(e.getSource() == saveButton) {
        String[][] result = new String[5][7];
        for (int i = 0; i < timeSlots.length; i++) {
            for (int j = 0; j < 7; j++) {
                if(scheduleTable.getValueAt(i,j+1).equals("Unavailable")) {
                    //if user unavailable, put 'N' in the txt file
                    result[i][j] = "N";
                }
                else if(scheduleTable.getValueAt(i, j+1).equals("Available")){
                    //if user available, put 'Y'
                    result[i][j] = "Y";
                }
                else {
                    //else uneditable, put 'X'
                    result[i][j] = "X";
                }
            }
        }
        //set folder and file names
        String foldername = date + " week";
        String filename = user.getEmail() + " " + date + " week";
        File folder = new File(foldername);
```

```java
        //find the path to folder
        //if folder doesn't exist, create one
        if(!folder.exists()) {
            folder.mkdir();
        }
        /*create a txt file for schedule data storage.
         * File name contains start of week date and username
         * so easy for system to fetch data later*/
        File file = new File(foldername + "/" + filename);
        try(FileWriter writer = new FileWriter(file)) {
            for(String[] row : result) {
                for(String cell : row) {
                    //use " " to separate values
                    writer.write(cell + " ");
                }
                // next line for next row
                writer.write("\n");
            }
            //display successful saved file
            JOptionPane.showMessageDialog(
                    null,
                    "File saved successfully",
                    "Success",
                    JOptionPane.INFORMATION_MESSAGE
                );
            //user try and catch for exception handling
        } catch (IOException e1) {
            e1.printStackTrace();
        }

    }
```

Figure 5.0 – ActionListener for save Button for saving schedule

6. Handling Week Data extraction:

In the class UserCurrentWeekSchedule, we would want to output the final document scheduled by the Admin. Therefore we will use the following to get the 2D array.

```
String foldername = date + " week";
String filename = foldername + "/" + date + " week final";
File filefinal = new File(filename);
//if week not scheduled
if (!filefinal.exists()) {
    //show error
    JOptionPane.showMessageDialog(
            this,
            "Week not scheduled",
            "Error",
            JOptionPane.ERROR_MESSAGE
    );
    //end the page
    this.dispose();
    return;
}
//process the scheduled file
String[][] data = processFile(filefinal);
//label the title of the page
titlelabel = new JLabel("Current week - Starting Date: " + date, JLabel.CENTER);
titlelabel.setFont(new Font("Serif", Font.BOLD, 24));
//table model configuration (Refer to Criterion B - User Current week page)
tablemodel = new DefaultTableModel(data.length, COLUMN_HEADERS.length + 1) {
    //Column_Headers.length + 1 because of headers
    @Override
    public String getColumnName(int column) {
        return (column == 0) ? "Meal Time" : COLUMN_HEADERS[column - 1];
    }

    @Override
    public boolean isCellEditable(int row, int column) {
        return false;
    }
};
```

```
    @Override
    public boolean isCellEditable(int row, int column) {
        //In this class for User, who are not allowed to edit
        //Refer to Criterion A - Success Criteria
        //disable edit for cell in the table
        return false;
    }
};
//enter data into JTable for visualize
for (int i = 0; i < data.length; i++) {
    tablemodel.setValueAt(ROW_HEADERS[i], i, 0);
    for (int j = 0; j < data[i].length; j++) {
        tablemodel.setValueAt(data[i][j], i, j + 1);
    }
}
```

Figure 6.0 – display table for User to view

The processFile() function takes in a file directory and return a 2D array collecting that specific txt file such as scheduled week:

```
private static String[][] processFile(File file) {
    List<String[]> lines = new ArrayList<>();
    /*use buffered reader to read the file line by line
     * and add them into the lines array that elements separated with " "*/
    //use try and catch function to handle exception
    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
        while ((line = reader.readLine()) != null) {
            lines.add(line.trim().split("\\s+"));
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
    //return array
    return lines.toArray(new String[0][]);
}
```

Figure 6.1 – processFile() method

For Admin's Frame of displaying current week, it is completely same as User's frame but allow Admin to

make changes to the plan, avoiding problems in scheduling (Refer to Criterion A – Successment Criteria):

```
tablemodel = new DefaultTableModel(data.length, COLUMN_HEADERS.length + 1) {
    @Override
    public String getColumnName(int column) {
        return (column == 0) ? "Meal Time" : COLUMN_HEADERS[column - 1];
    }

    @Override
    public boolean isCellEditable(int row, int column) {
        return column != 0;
    }
};
```

Allow Admin to Edit the table on certain boxes

Figure 6.2 – Table construction for Admin

7. Allow Admin to output schedule as a HTML file:

Additional function for Admin is to output the schedule as an HTML file. It should allow user to save at any place on the computer:

```
if(e.getSource() == Schedule_to_picture) {
    //similar to the start of week date calculation mentioned in Figure 2.1
    LocalDate today = LocalDate.now();
    DayOfWeek dayinweek = today.getDayOfWeek();
    int daynumber = dayinweek.getValue();
    LocalDate startofthisweek = today.minusDays(daynumber - 1);
    String date = startofthisweek.toString();
    String foldername = date + " week";
    String filename = foldername + "/" + date + " week final";
    File folder = new File(foldername);
    File file = new File(filename);
    //if final file exist(week scheduled)
    if(folder.exists() && file.exists()) {
        String[][] timetable = processFile(new File(filename));
        //create a StringBuilder from java.lang
        StringBuilder html = new StringBuilder();
        //a JFileChooser for Admin to save the HTML file
        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setDialogTitle("Save");
        fileChooser.setFileFilter(new FileNameExtensionFilter("HTML Files", "html"));
```

```java
/*defines HTML document structure and styles the table with boardered cells,
 * padding and centered text*/
html.append("<!DOCTYPE html>\n<html>\n<head>\n<title>Schedule</title>\n");
html.append("<style>table {border-collapse: collapse; width: 50%;}");
html.append("th, td {border: 1px solid black; padding: 8px; text-align: center;}</style>\n");
html.append("</head>\n<body>\n");
//Add title, row and column headers
html.append("<h2>Schedule week " + date +" </h2>\n<table>\n");
html.append("<tr>");
for (String colHeader : COLUMN_HEADERS) {
    html.append("<th>").append(colHeader).append("</th>");
}
html.append("</tr>\n");
/*fill the table with data: this program iterate through each row.
 * The first column contains meal time labels and remaining column contain
 * scheduled data*/
for (int i = 0; i < timetable.length; i++) {
    html.append("<tr>");
    html.append("<th>").append(ROW_HEADERS[i]).append("</th>");
    for (String cell : timetable[i]) {
        html.append("<td>").append(cell).append("</td>");
    }
    html.append("</tr>\n");
}
//close the html document by ending
html.append("</table>\n</body>\n</html>");
```
[4]

[4] Khan, Muzammil. "Create HTML Table in Java." Aspose Blog. Aspose Blog | File Format Processing APIs for C# .NET Core, Java, C++, Python, December 28, 2023. https://blog.aspose.com/html/create-html-table-in-java.

```java
        //open file dialogue
        int userSelection = fileChooser.showSaveDialog(null);
        if (userSelection == JFileChooser.APPROVE_OPTION) {
            //Retrieves the files path chosen by user and change the file name with .html ending
            File fileToSave = fileChooser.getSelectedFile();
            String filePath = fileToSave.getAbsolutePath();
            if (!filePath.toLowerCase().endsWith(".html")) {
                filePath += ".html";
            }
            /*Uses Filewriter to write HTML content to the file
             * and use try and catch to ensure file choose properly*/
            try (FileWriter writer = new FileWriter(filePath)) {
                writer.write(html.toString());
            } catch (IOException e1) {
                e1.printStackTrace();
            }
        } else {
            System.out.println("Save operation was canceled.");
        }

    }
    else {
        //display error when week is not scheduled yet
        JOptionPane.showOptionDialog(
                null,
                "Week not scheduled",
                "error",
                JOptionPane.DEFAULT_OPTION,
                JOptionPane.ERROR_MESSAGE,
                null,
                new Object[] { "OK" },
                "OK"
            );
    }
```

Figure 7.0 – HTML construction for output

8. Schedule Timetable:

The admin should be able to schedule week from user's data when clicked the "Schedule current week" button:

```java
@Override
public void actionPerformed(ActionEvent e) {
    //admin click on schedule
    if(e.getSource() == Schedule_current_week) {
        //get current week (any date) and calculate start date
        LocalDate today = LocalDate.now();
        DayOfWeek dayinweek = today.getDayOfWeek();
        int daynumber = dayinweek.getValue();
        LocalDate startofweek = today.minusDays(daynumber - 1);
        String startofcurrentweek = startofweek.toString();
        //schedule (Figure 8.3)
        String[][] result = schedulefinalarray(startofcurrentweek);
        //display success schedulling
        JOptionPane.showMessageDialog(null,
                "File scheduled successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
```

Figure 8.0 – ActionListener for Schedule_current_week button

```java
private String[][] schedulefinalarray(String startOfWeekDate){
    //check if that week have already scheduled
    String filename = startOfWeekDate + " week final";
    String foldername = startOfWeekDate + " week";
    File folder = new File(foldername);
    File filefinal = new File(foldername + "/" + filename);
    if(!filefinal.exists()) {
        //if not scheduled
        //get all the file exists in the folder
        //and create a HashMap to store data for each email/Username
        File[] files = folder.listFiles();
        Map<String, String[][]> emailToDataMap = new HashMap<>();
        //loop through all files in the folder
        //if the file contains the start of week date then:
        //1. extract the email from name using extractEmailFromFileName() method
        //(in figure 8.2)
        //2. Read schedule data by user
        //3. Stores data into the HashMap
        for (File file : files) {
            if (file.getName().contains(startOfWeekDate)) {
                String email = extractEmailFromFileName(file.getName());
                String[][] data = processFile(file);
                emailToDataMap.put(email, data);
            }
        }
    //call organizeDataForLocations() method to combine into 1 final schedule
    //shown in figure 8.3
    String[][] finalArray = organizeDataForLocations(emailToDataMap);
    //write the final schedule 2D array into the final file and return it
    try(FileWriter writer = new FileWriter(filefinal)) {
        for(String[] row : finalArray) {
            for(String cell : row) {
                writer.write(cell + " ");
            }
            writer.write("\n");
        }
    } catch (IOException e1) {
        e1.printStackTrace();
    }
    return finalArray;
    }
//if the file exist (already scheduled) then we don't have to reschedule
//return the original schedulled data.
else {
    String[][] finalArray = processFile(filefinal);
    return finalArray;
}
```

Figure 8.1 – Schedulefinalarray() method

The function processFile() is the same as Figure 6.1.

```
private static String extractEmailFromFileName(String fileName) {
    /*define a regular expression:
     * ^ means start of string
     * (.*?) means capture the shortest sequence of any characters
     * (.*?) and \s+ ends at the first space (one or more spaces)*/
    Pattern pattern = Pattern.compile("^(.*?)\\s+");
    //matcher to apply pattern to filename
    Matcher matcher = pattern.matcher(fileName);
    //return everything before 1st space
    if (matcher.find()) {
        return matcher.group(1);
    }
    return "";
}
```

Figure 8.2 – extractEmailFromFileName() method

Schedule method:

```
private static String[][] organizeDataForLocations(Map<String, String[][]> emailToDataMap) {
    //determines the size of the schedule by iterating through each person's availability of data.
    int numRows = 0, numCols = 0;
    for (String[][] data : emailToDataMap.values()) {
        numRows = data.length;
        numCols = data[0].length;
    }
    //creates a final 2D array for final schedule starting everything with " "
    String[][] finalArray = new String[numRows][numCols];
    for (int i = 0; i < numRows; i++) {
        for (int j = 0; j < numCols; j++) {
            finalArray[i][j] = " ";
        }
    }
    //this map assignmentCount keep track of how many times each email is assigned and set to 0 initially
    //(Refer to Criterion A - Success Critera)
    Map<String, Integer> assignmentCount = new HashMap<>();
    for (String email : emailToDataMap.keySet()) {
        assignmentCount.put(email, 0);
    }
```
[5]

[5] Interview Cake: Programming Interview Questions and Tips. “Hash Table/Hash Map Data Structure | Interview Cake,” n.d. https://www.interviewcake.com/concept/java/hash-map.

```java
//creates a Map<Integer, List<String>> to associate each location
//(column index with the people available to that location)
Map<Integer, List<String>> locationToEmails = new HashMap<>();
for (Map.Entry<String, String[][]> entry : emailToDataMap.entrySet()) {
    String email = entry.getKey();
    String[][] data = entry.getValue();
    for (int j = 0; j < numCols; j++) {
        for (int i = 0; i < numRows; i++) {
            if ("X".equals(data[i][j])) {
                //if the cell is "X", the same "X" copied to the final array
                //because it indicate that that location doesn't need to be scheduled
                finalArray[i][j] = "X";
            } else if ("Y".equals(data[i][j])) {
                //if "Y" then the email is added to the locationToEmails:
                //meaning that they are available at that specific location
                locationToEmails.computeIfAbsent(j, k -> new ArrayList<>()).add(email);
            }
        }
    }
}
```

```java
/*convert locationToEmails into a list of entries and sort it by number of people available in
 * ascending order (ensures that locations with fewer available space are filled first, minimizing
 * schedulling conflicts)*/
List<Map.Entry<Integer, List<String>>> sortedLocations = new ArrayList<>(locationToEmails.entrySet());
sortedLocations.sort(Comparator.comparingInt(e -> e.getValue().size()));
/*priority queue is used to distribute work evenly.
 * The queue is sorted based on assignmentCount,
 * ensuring that the least-assigned person is selected first*/
PriorityQueue<String> emailQueue = new PriorityQueue<>(Comparator.comparingInt(assignmentCount::get));
/*loop through each location in sorted order and gets the column index (locationindex)
 * and the list of people available for that location.
 * It also clears the emailQueue and refills it with available people for that location*/
for (Map.Entry<Integer, List<String>> locationEntry : sortedLocations) {
    int locationIndex = locationEntry.getKey();
    List<String> emailsAtLocation = locationEntry.getValue();
    emailQueue.clear();
    emailQueue.addAll(emailsAtLocation);
    //iterate through all rows to assign people at given location and skip location for "X"
    for (int i = 0; i < numRows; i++) {
        if (!"X".equals(finalArray[i][locationIndex])) {
            /*Polls the least-assigned person from the queue and check if they are available
             * at that specific location. If so, assign them into the final array and increase their
             * assignment count. Breaks the loop once a person is assigned, and move onto the next row*/
            while (!emailQueue.isEmpty()) {
                String email = emailQueue.poll();
                if ("Y".equals(emailToDataMap.get(email)[i][locationIndex])) {
                    finalArray[i][locationIndex] = email;
                    assignmentCount.put(email, assignmentCount.get(email) + 1);
                    break;
                }
            }
            /*if no one is assigned in a location, this means that no one is available
             * set the cell to "X" which enable Admin to edit that spot*/
            if (finalArray[i][locationIndex].equals(" ")) {
                finalArray[i][locationIndex] = "X";
            }
```

6

Figure 8.3 – organizeDataForLocations() method

In sum, this the function **organizeDataForLocations** distributes people as evenly as possible across a schedule while handling locations that do not require assignments (`"X"`) correctly. It first determines the schedule's size (`numRows`, `numCols`) and initializes a final array. It then maps each location (column) to the people available for that spot. Locations are sorted in ascending order of available people to prioritize filling the most constrained spots first. A priority queue is used to ensure assignments are balanced, always selecting the least-assigned person whenever possible. Assignments are made row by row, ensuring that all available spots are filled while maintaining fairness. If no one is available for a location, it remains marked as `"X"`. This approach ensures efficient and even distribution of people across the schedule.

9. Libraries used:

```
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;
import javax.swing.border.EmptyBorder;
import javax.swing.filechooser.FileNameExtensionFilter;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
```

Frame layout, font and dimension settings

User events(clicking for buttons, focus, etc.)

box layout

buttons

File chooser for Admin

Frame superclass

label(titles or text in frame)

output information for user

Jpanel in frame supports format and other components

Textfields

file chooser

construct table

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.text.SimpleDateFormat;
import java.time.DayOfWeek;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
```

handle program and file communication

handle exception

date handling

additional dynamic data structure

recognise pattern in file names

Word count: 1074 words

Bibliography:

Calpoly.edu. "TableCellRenderer Examples," 2025. https://users.csc.calpoly.edu/~jdalbey/305/Lectures/TableCellRendererExamples.html.

Code, Bro. "Java Full Course ☕ (□□□□)." www.youtube.com, November 9, 2020. https://www.youtube.com/watch?v=xk4_1vDrzzo.

GeeksforGeeks. "Java.io.BufferedWriter Class Methods in Java." GeeksforGeeks, December 29, 2016. https://www.geeksforgeeks.org/io-bufferedwriter-class-methods-java.

Interview Cake: Programming Interview Questions and Tips. "Hash Table/Hash Map Data Structure | Interview Cake," n.d. https://www.interviewcake.com/concept/java/hash-map.

Justiciar. "Creating a Custom Table Model to Provide Data from ArrayList." Stack Overflow, March 30, 2017. https://stackoverflow.com/questions/43106351/creating-a-custom-table-model-to-provide-data-from-arraylist..

Khan, Muzammil. “Create HTML Table in Java.” Aspose Blog. Aspose Blog | File Format Processing APIs for C# .NET Core, Java, C++, Python, December 28, 2023. https://blog.aspose.com/html/create-html-table-in-java.

Oracle Help Center. “JDK 24 Documentation - Home,” 2025. https://docs.oracle.com/javase/8/docs/api/javax/swing/table/TableCellRenderer.html.

Oracle.com. “Java Platform SE 8,” 2025. https://docs.oracle.com/javase/8/docs/api///?javax/swing/DefaultCellEditor.html.

Oracle.com. “Overview (Java Platform SE 8),” September 11, 2019. https://docs.oracle.com/javase/8/docs/api/overview-summary.html.

W3schools. “Java Tutorial.” W3schools.com, 2019. https://www.w3schools.com/java/default.asp.

www.digitalocean.com. “Java FileWriter Example | DigitalOcean,” n.d. https://www.digitalocean.com/community/tutorials/java-filewriter-example.

www.w3schools.com. “Java Files,” n.d. https://www.w3schools.com/java/java_files.asp.

.